

Machine Learning with Python Project Documentation

Angel Siwele

NS.2022.R4G0J9

Project A

06/11/2023

System Requirements

To run the ML model, the PC you will be running the program on must have the following installed:

1. Python
2. Any Python Friendly IDE such as: Visual Studio Code, PyCharm Community or Python IDLE .

Installation and Setup

1. Install Python
 - a. Visit this link (<https://www.python.org/downloads/>) and download the latest version of Python. Once downloaded, Double Click to Install on PC.
2. Install one of the following Python friendly IDE's
 - a. Python IDLE is automatically installed when you install Python.
 - b. To install Visual Studio Code visit this link <https://code.visualstudio.com/> and download the latest version of Visual Studio Code. Once downloaded, double click to Install on PC
 - c. To install PyCharm Community visit this link (<https://www.jetbrains.com/pycharm/download/?section=windows>) and download the latest version of PyCharm Community. Once downloaded, double click to Install on PC
3. Once the above have been installed, you can run the model on the IDE.

Instructions

After unzipping the folder submitted as 'Machine Learning Project', right click on the 'Version A' file and select 'open with' to choose your preferred Python IDE. Once the code file is open, run the model by clicking the play button or right click and select 'Run Version A'. The terminal will appear and ask you to input various features such as: 'Gender', 'Fatigued', 'Raining', 'Alcohol Consumed', and 'Time of Day'. The model will then calculate and display the probability of a road accident based on the features you enter..

Source Code

```
"""
Author: Angel Siwele
Date: 2023/11/06
Scenario:
The high amount of road accidents that occur is a major concern for the Department of Roads and Transportation.
They would like to make use of emerging technology to predict the probability of road accidents. You have been
contracted to provide three machine learning models that are capable of predicting the probability of a road accident,
based on a variety of factors at the time of the accident
"""

import pickle
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import random

from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import LabelBinarizer
from sklearn.preprocessing import OneHotEncoder
from sklearn.svm import SVC
from sklearn.naive_bayes import CategoricalNB
from sklearn.neighbors import KNeighborsClassifier

### Question 1 ###
# Create a Toy Dataset
def create_toy_data():
    num_rows = 1000
```

```

data = {
    'Gender': [random.choice(['Male', 'Female']) for _ in range(num_rows)],
    'Age': [random.randint(18, 70) for _ in range(num_rows)],
    'Alcohol Consumed': [random.choice(['Yes', 'No']) for _ in range(num_rows)],
    'Fatigued': [random.choice(['Yes', 'No']) for _ in range(num_rows)],
    'Time of day': [random.choice(['Daylight', 'Nighttime']) for _ in range(num_rows)],
    'Raining': [random.choice(['Yes', 'No']) for _ in range(num_rows)]
}

# Create a DataFrame from the custom data
custom_df = pd.DataFrame(data)

# Generate the target variable using make_classification
features, target = make_classification(
    n_samples=num_rows,
    n_features=6, # Set this to 6 to match the number of features in the custom data (excluding the target)
    n_informative=6, # Set this to 6 to match the number of informative features in the custom data
    n_redundant=0,
    n_classes=2,
    weights=[0.8, 0.2],
    random_state=42
)

# Map target variable values from binary (0 and 1) to 'Yes' and 'No'
target = ['Yes' if val == 1 else 'No' for val in target]

# Add the modified target variable to the DataFrame
custom_df['Accident'] = target

#print(custom_df)

# Export the DataFrame to a CSV file
custom_df.to_csv('road_accidents_toy_dataset.csv', index=False)

#create_toy_data()

# Load csv into DataFrame
df = pd.read_csv('road_accidents_toy_dataset.csv')

## Preprocess Data ##

```

```

# Encoding features and target
label_encoder = LabelEncoder()
label_binarizer = LabelBinarizer()
label_one_hot_encoder = OneHotEncoder()
df['Fatigued'] = label_binarizer.fit_transform(df['Fatigued'])
df['Raining'] = label_binarizer.fit_transform(df['Raining'])
df['Alcohol Consumed'] = label_binarizer.fit_transform(df['Alcohol Consumed'])
df['Gender'] = label_binarizer.fit_transform(df['Gender'])
df['Time of day'] = label_encoder.fit_transform(df['Time of day'])
df['Accident'] = label_binarizer.fit_transform(df['Accident'])
df.to_csv('clean_road_accidents_toy_dataset.csv', index=False)
#print(df)

#### Question 2 ####
# Train a Machine Learning Model

x = df.drop(['Accident'], axis=1)
y = df['Accident']

# Set a random seed for reproducibility
random_seed = 40
# Split data into training and testing sets
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=random_seed)

clf1 = LogisticRegression()
clf1.fit(x_train, y_train)
print(f'Log Regression Accuracy: {clf1.score(x_test, y_test)}')

# Collect input from the user
predict_gender = input('Gender (Male/Female): ')
predict_age = input('Age: ')
predict_alcohol_consumed = input('Alcohol Consumed (Yes/No): ')
predict_fatigued = input('Fatigued (Yes/No): ')
predict_time_of_day = input('Time of Day (Daylight/Nighttime): ')

```

```

predict_raining = input('Raining (Yes/No): ')

# Create a dictionary with the user input
predict = {
    'Gender': predict_gender,
    'Age': predict_age,
    'Alcohol Consumed': predict_alcohol_consumed,
    'Fatigued': predict_fatigued,
    'Time of day': predict_time_of_day,
    'Raining': predict_raining
}

# Convert the dictionary to a Pandas Series
pred = pd.Series(predict)
# Print the input data
#print(pred)

# Mapping 'Yes' to 1 and 'No' to 0 for binary variables
pred['Fatigued'] = 1 if pred['Fatigued'] == 'Yes' else 0
pred['Raining'] = 1 if pred['Raining'] == 'Yes' else 0
pred['Alcohol Consumed'] = 1 if pred['Alcohol Consumed'] == 'Yes' else 0
pred['Gender'] = 1 if pred['Gender'] == 'Male' else 0
pred['Time of day'] = 1 if pred['Time of day'] == 'Nighttime' else 0
#print(pred)

# use input samples to find probability estimates
probability_estimates = clf1.predict_proba(pred.to_numpy().reshape(1, -1))
# Probability estimates for the positive class (class 1)
probability_estimates_class_1 = probability_estimates[:, 1]
print(f'Probability of Vehicle Accident: {probability_estimates_class_1}')

"""
clf2 = DecisionTreeClassifier()

```

```
clf2.fit(x_train, y_train)
print(f'Tree Accuracy: {clf2.score(x_test, y_test)}')

clf3 = RandomForestClassifier()
clf3.fit(x_train, y_train)
print(f'Random Forrest Accuracy: {clf3.score(x_test, y_test)}')

clf4 = SVC()
clf4.fit(x_train, y_train)
print(f'SVC Accuracy: {clf4.score(x_test, y_test)}')

clf5 = CategoricalNB()
clf5.fit(x_train, y_train)
print(f'Naive Bayes Accuracy: {clf5.score(x_test, y_test)}')

clf6 = KNeighborsClassifier()
clf6.fit(x_train, y_train)
print(f'KNN Accuracy: {clf6.score(x_test, y_test)}')
'''
```

Process of Creating the Machine Learning Model

1. Dataset Creation:

A synthetic dataset comprising 1000 rows was generated to simulate real-world scenarios. The dataset includes features such as gender, age, alcohol consumption, fatigue, time of day, and weather conditions. The target variable, 'Accident,' was created using the `make_classification` function from `scikit-learn`. The (raw) dataset was exported to a CSV file, 'road_accidents_toy_dataset.csv'. This is done in the Question 1 Section of the code.

2. Data Preprocessing:

The dataset underwent preprocessing to prepare it for machine learning models. Categorical variables were encoded using `LabelEncoder`, `LabelBinarizer`, and `OneHotEncoder`. The processed data was saved to a new CSV file, 'clean_road_accidents_toy_dataset.csv.' This is also done in the Question 1 Section of the code.

3. Model Training:

A logistic regression model was chosen as the initial predictive model. The dataset was split into training and testing sets, and the model achieved a commendable accuracy of [insert accuracy here] on the test set. The code includes other classifier models such as Decision Tree, Random Forest, Support Vector Classifier (SVC), Naive Bayes, and K-Nearest Neighbors, although they are commented out. To compare model performances, it is recommended to uncomment and evaluate these models. This is done in the Question 2 part of the code.